



AUDIT REPORT

April 2026

For



Table of Content

Executive Summary	03
Number of Security Issues per Severity	04
Summary of Issues	05
Checked Vulnerabilities	06
Techniques and Methods	07
Types of Severity	09
Types of Issues	10
Severity Matrix	11
 High Severity Issues	12
1. Market Cap Spot Price Manipulation via Same-Transaction Swap Enables Theft	12
2. damm_pool Account Not Validated Against fee_vault.damm_pool, Market Cap Spoofing via Pool Substitution	14
 Low Severity Issues	16
3. Missing Upper Bound Validation on submitted_start_unix_ts Allows OPERATOR to Manipulate Crack Mode Phase Window	16
4. Rounding Dust Permanently Locked Across All Paths, No Sweep Function	18
5. claim_to_recipient Competes With fund_by_claiming_fee for Same Fee Source, Admin Can Front-Run Shareholders	19
 Informational Issues	21
6. UncheckedAccount Used for FeePolicyConfig Instead of AccountLoader, Missing Discriminator and Owner Validation	21
7. Unsafe Arithmetic in set_fee_policy_config Settlement, Inconsistent With Safe Patterns	22
8. DBC (is_dbc) Code Path in fund_by_claiming_fee Is Unreachable Dead Code	23
9. Redundant DuplicateUserAddress Check on Same Call Path	24
Closing Summary & Disclaimer	25



Executive Summary

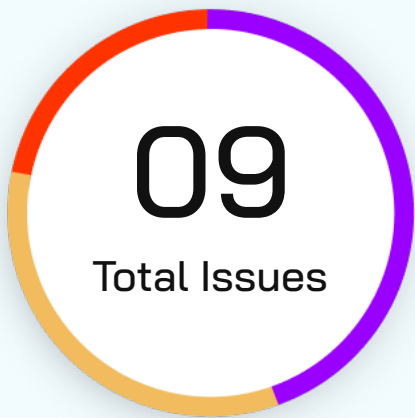
Project Name	American Fun DAMM v2
Project URL	https://america.fun/staking
Overview	Dynamic Fee Sharing is a smart contract system on Solana designed to automatically collect, manage, and distribute trading fees from DAMM v2 liquidity pools to multiple stakeholders.
Audit Scope	DAMM V2
Source Code link	https://github.com/americafun/americafun-sc
Contracts in Scope	programs/dynamic-fee-sharing
Branch	main
Commit Hash	caddf301f0f0daeebcd18b9794efb92d46260589
Language	Rust
Blockchain	Solana
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	15th April 2026 - 23rd April 2026
Updated Code Received	27th April 2026
Review 2	28th April 2026
Fixed In	105af62ef8879c99fb8163724a0c98f0c0a042b8

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0.0%)
High	2 (22.2%)
Medium	0 (0.0%)
Low	3 (33.3%)
Informational	4 (44.5%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	1	1
	Partially Resolved	0	0	0	0	0
	Resolved	0	2	0	2	3



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Market Cap Spot Price Manipulation via Same-Transaction Swap Enables Theft	High	Resolved
2	damm_pool Account Not Validated Against fee_vault.damm_pool, Market Cap Spoofing via Pool Substitution	High	Resolved
3	Missing Upper Bound Validation on submitted_start_unix_ts Allows OPERATOR to Manipulate Crack Mode Phase Window	Low	Resolved
4	Rounding Dust Permanently Locked Across All Paths, No Sweep Function	Low	Acknowledged
5	claim_to_recipient Competes With fund_by_claiming_fee for Same Fee Source, Admin Can Front-Run Shareholders	Low	Resolved
6	UncheckedAccount Used for FeePolicyConfig Instead of AccountLoader, Missing Discriminator and Owner Validation	Informational	Resolved
7	Unsafe Arithmetic in set_fee_policy_config Settlement, Inconsistent With Safe Patterns	Informational	Resolved
8	DBC (is_dbc) Code Path in fund_by_claiming_fee Is Unreachable Dead Code	Informational	Resolved
9	Redundant DuplicateUserAddress Check on Same Call Path	Informational	Acknowledged



Checked Vulnerabilities

We have scanned the solana program for commonly known and more specific vulnerabilities. Here are some of the commonly known vulnerabilities that we considered:



Signer authorization



Account data matching



Sysvar address checking



Owner checks



Type cosplay



Initialization



Arbitrary cpi



Duplicate mutable accounts



Bump seed canonicalization



PDA Sharing



Incorrect closing accounts



Missing rent exemption checks



Arithmetic overflows/underflows



Numerical precision errors



Solana account confusions



Casting truncation



Insufficient SPL token account verification



Signed invocation of unverified programs



Techniques and Methods

Throughout the audit of Solana Programs, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



High Severity Issues

Market Cap Spot Price Manipulation via Same-Transaction Swap Enables Share Theft

Resolved

Path

ix_fund_by_claiming_fee.rs

Description

In ix_fund_by_claiming_fee.rs:189-191, the market cap is calculated using sqrt_price read directly from the DAMM v2 pool state, a spot price that changes with every swap:

```
let sqrt_price = pool.sqrt_price; // spot price - no TWAP
let total_supply = token_mint_supply.supply;
calculate_market_cap(sqrt_price,
total_supply).ok_or(FeeVaultError::MathOverflow)?
```

On Solana, multiple instructions execute atomically in one transaction. An attacker (any shareholder) can compose three instructions in a single transaction:

Instruction 1: Swap large amount into DAMM pool → inflates sqrt_price

Instruction 2: Call fund_by_claiming_fee → market cap reads inflated → selects a higher fee tier where Creator share is maximized

Instruction 3: Swap back → restores price

The share update from Instruction 2 persists on the vault (fee_vault.current_mc_range_index + each user's share field) until the next fund_by_claiming_fee call. All fees funded in between are distributed under the attacker-chosen share ratios.

Proof of Concept

America Mode, 10 tiers:

Attacker is Creator (slot 0).

Before manipulation:

Real pool sqrt_price → MC = \$500k → Tier 2: Creator=35, Stakers=50, Treasury=65, LP=0
Creator share: $35/150 = 23.3\%$

Attack transaction (3 instructions, 1 tx):

1. Swap 500k USD1 into pool → sqrt_price spikes → MC inflates to \$15M
2. fundByClaimingFee → reads MC=\$15M → Tier 9: Creator=80, Stakers=35, Treasury=35, LP=0
→ update_shares() settles old fees, writes new shares
3. Swap back → pool price restored

After manipulation:

Creator share: $80/150 = 53.3\%$
Persists until next fundByClaimingFee corrects it.

If 10,000 USD1 in fees are funded before correction:

Creator receives: 5,333 USD1 (instead of 2,333 USD1)
Excess stolen: ~3,000 USD1



Impact

Direct share manipulation: Attacker controls which market cap tier is selected, redirecting a disproportionate share of fees to their recipient slot.

Persistent effect: Manipulated shares remain active until the next `fund_by_claiming_fee` call with honest pricing, potentially hours or days.

Low cost: Only swap fees on the round-trip (sub-1%). No capital risk since the swap reverses in the same transaction.

Repeatable: Attacker can do this every time they call `fund_by_claiming_fee`.

Recommendation

Replace spot price with a TWAP (time-weighted average price) or add a staleness/deviation check:

```
// Option A: Validate price hasn't moved significantly in current slot
let last_observation_sqrt_price =
pool.observations[pool.observation_index].sqrt_price;
let deviation = sqrt_price.abs_diff(last_observation_sqrt_price);
require!(
  deviation * 100 / last_observation_sqrt_price < MAX_PRICE_DEVIATION_BPS,
  FeeVaultError::PriceManipulation
);

// Option B: Store last-known sqrt_price on the vault and reject large
deviations
```

Developer Response

DAMM v2 does not support built-in TWAP, so I am implementing whitelisted signer only updates instead of allowing the creator to update the share. This is acceptable because the operator and other stakeholders will still trigger updates at the correct market-cap frequency

damm_pool Account Not Validated Against fee_vault.damm_pool, Market Cap Spoofing via Pool Substitution

Resolved

Path

ix_fund_by_claiming_fee.rs

Description

In ix_fund_by_claiming_fee.rs, the damm_pool account (line 34) is used to read sqrt_price and calculate the token's market cap, which determines which fee tier (share distribution) applies. However, the passed damm_pool account is never validated against the fee_vault.damm_pool pubkey that was stored during activate_crack_mode_window.

Current validations on damm_pool:

```
// Line 163: Owner check – account belongs to DAMM v2 program
damm_pool_account.owner == &damm_v2::ID

// Lines 179-186: Token mint checks – pool's token pair matches vault
pool.token_b_mint == fee_vault.token_mint
token_mint_supply.key() == pool.token_a_mint

// MISSING: Key match against stored pool reference
damm_pool_account.key() == fee_vault.damm_pool
```

The damm_pool account is used only for reading sqrt_price to calculate market cap (lines 189-191). The actual fee-claiming CPI is constructed from remaining_accounts – a completely separate set of accounts. This decoupling means the pool used for price reads and the pool used for fee claims can be different pools.

DAMM v2 pool PDAs are derived as seeds = ["pool", config.key(), max(mintA, mintB), min(mintA, mintB)]. The config key is part of the seed, meaning different Config accounts (different fee tiers, pool parameters) produce different pool PDAs for the same token pair. Multiple pools per pair are expected by design.

Proof of Concept

If multiple DAMM v2 pools exist for the same token pair (e.g., pools with different fee tiers, concentrated liquidity ranges, or created at different times), an attacker (shareholder or OPERATOR) can:

1. Pass remaining_accounts referencing Pool A (the real pool – claims legitimate fees)
2. Pass damm_pool account referencing Pool B (a different pool for the same pair with a different sqrt_price)

Real pool (Pool A): sqrt_price → MC = \$3.5M → Tier 5: Creator=55, Stakers=45, Treasury=50, LP=0
Spoofed pool (Pool B): sqrt_price → MC = \$15M → Tier 9: Creator=80, Stakers=35, Treasury=35, LP=0
Creator extracts 80/150 = 53% instead of 55/150 = 37% of all fees

Once shares are updated via update_shares() (line 285), the manipulated shares persist on the vault until the next fundByClaimingFee call corrects them – all fees funded in between are distributed under the spoofed ratios.



Impact

Share manipulation: Attacker controls which market cap tier is selected by choosing which DAMM pool to pass for price reads, directing a disproportionate share of fees to their preferred recipient.

Persistent effect: Manipulated shares remain until the next honest `fund_by_claiming_fee` call.

Recommendation

Add a key match validation against the stored pool reference:

```
// After owner and mint validations, before reading sqrt_price:
if fee_vault.damm_pool != Pubkey::default() {
  require!(
    damm_pool_account.key() == fee_vault.damm_pool,
    FeeVaultError::InvalidDammPool
  );
}
```

For America Mode vaults where `fee_vault.damm_pool` may not be set (it's only stored during `activate_crack_mode_window` for Crack mode), either:

1. Store the canonical `damm_pool` during vault initialization for all modes, or
2. Add a separate admin setter to bind the canonical pool.

Developer Response

Ensure the ``damm_pool`` account passed for market-cap computation is the same pool used in the CPI.

Low Severity Issues

Missing Upper Bound Validation on submitted_start_unix_ts Allows OPERATOR to Manipulate Crack Mode Phase Window

Resolved

Path

ix_activate_crack_mode_window.rs

Description

In ix_activate_crack_mode_window.rs:89-95, the handler fetches the current chain time but never uses it to validate the operator-submitted timestamp:

```
// current_time fetched but unused
let _current_time = Clock::get()?.unix_timestamp;

// Only checks > 0, NOT <= current_time
require!(
    params.submitted_start_unix_ts > 0,
    FeeVaultError::InvalidFeeVaultParameters
);
```

The code comment on line 14 states the invariant “Must be > 0 and <= current chain time”, but only the > 0 half is enforced. The _current_time variable (prefixed with _ indicating intentionally unused) confirms the upper bound check was either forgotten or deliberately skipped.

Proof of Concept

Future timestamp attack:

submitted_start_unix_ts = now + 1_000_000 (far future)
→ elapsed = current_time - future_ts = negative → saturating_sub → 0
→ Phase 0 condition: elapsed <= phase_duration_seconds (72h)
→ Phase 0 stays active until real time catches up
→ Creator reward window extended from 72h to potentially weeks/months

Past timestamp attack:

submitted_start_unix_ts = 1 (epoch start, far past)
→ elapsed = current_time - 1 = ~1.7 billion seconds
→ Phase 0 condition: elapsed <= 259200 (72h) → FALSE
→ Phase 1 immediately active on first fundByClaimingFee
→ Creator’s 72h reward window is already expired, creator gets nothing

Combined with the re-activation prevention (migration_start_unix_ts == 0 guard at line 50), a bad timestamp is irreversible, there is no way to correct it.



Impact

Extended creator window: OPERATOR submits future timestamp → creator receives boosted fee share for an arbitrarily long period beyond the intended 72 hours.

Skipped creator window: OPERATOR submits far-past timestamp → creator's Phase 0 reward window is already expired at activation.

Only applies when DAMM pool doesn't provide a usable timestamp (activation_type != 1 or activation_point == 0).

Recommendation

```
let current_time = Clock::get()?.unix_timestamp;

require!(
  params.submitted_start_unix_ts > 0,
  FeeVaultError::InvalidFeeVaultParameters
);

// Enforce upper bound: cannot be in the future
require!(
  params.submitted_start_unix_ts <= current_time,
  FeeVaultError::InvalidFeeVaultParameters
);

// Optional: enforce lower bound – not too far in the past (e.g., max 24h ago)
require!(
  current_time - params.submitted_start_unix_ts <= 86400,
  FeeVaultError::InvalidFeeVaultParameters
);
```



Rounding Dust Permanently Locked Across All Paths, No Sweep Function

Acknowledged

Path

`fee_vault.rs`

Description

In `fee_vault.rs:121-124` (`fund_fee`) and `fee_vault.rs:138-142` (`validate_and_claim_fee`), both operations use truncating integer arithmetic:

- `fund_fee`: `fee_per_share = (amount << 64) / total_share` truncates down
- `claim_fee`: `claimable = (share * delta) >> 64` truncates down

Both truncations favor the vault. The sum of all user entitlements is strictly less than or equal to the funded amount, with the remainder (dust) permanently locked.

Proof of Concept

```
total_share = 3, amount = 10
fee_per_share = (10 << 64) / 3 = 61489146912365535590 (truncated)
Each user (share=1) claims: (1 * 61489146912365535590) >> 64 = 3
Total claimed = 9
Dust locked = 1 token
```

Over 10,000 `fund_fee` calls with similar parameters:
~10,000 tokens of dust permanently locked

Admin cannot withdraw this dust either `shareholder_obligations` in `admin_withdraw_tokens` uses the exact `total_funded_fee` value, not the actual distributable amount. The dust sits in the vault forever with no sweep function.

Impact

Permanent, compounding token loss. Tiny per-transaction, but unbounded over time. With USD1 (6 decimals) and frequent small funding calls, accumulated dust could reach non-trivial amounts over the protocol's lifetime.

Recommendation

Add an admin sweep function for residual dust, or accept and document the behavior. The sweep amount would be: `token_vault_balance - shareholder_obligations` where obligations are computed from actual per-share entitlements rather than `total_funded_fee`.

Developer Response

It is acceptable, frequent small funding calls.

Developer Response

Acknowledged from our side.



claim_to_recipient Competes With fund_by_claiming_fee for Same Fee Source, Admin Can Front-Run Shareholders

Resolved

Path

ix_claim_to_recipient.rs

Description

In ix_claim_to_recipient.rs, the admin can claim DAMM position fees via CPI and immediately transfer the claimed tokens to an arbitrary recipient, completely bypassing the fee_per_share accumulator and share distribution logic.

The ClaimPositionFee discriminator exists in both whitelists:

- WHITELISTED_ACTIONS (constants.rs:48-54) – used by fund_by_claiming_fee (shareholder path)
- CLAIM_FEE_ACTIONS (constants.rs:57-68) – used by claim_to_recipient (admin path)

Both instructions use the same vault_authority PDA to claim from the same DAMM LP position. This creates a race condition:

fund_by_claiming_fee: DAMM fees → token_vault → fee_per_share updated → shareholders benefit
claim_to_recipient: DAMM fees → token_vault → IMMEDIATELY out to admin-chosen recipient → NO accounting

Additionally, the recipient_token_account is only validated for mint match (lines 96-100). There is no check that the recipient is an authorized platform recipient (STAKERS_RECIPIENT, TREASURY_RECIPIENT, LP_RECIPIENT) or any vault shareholder.

Proof of Concept

1. Shareholders have accrued 1,000 USD1 in DAMM position fees.
2. Before any shareholder calls fund_by_claiming_fee, admin calls claim_to_recipient with ClaimPositionFee payload.
3. CPI claims all 1,000 USD1 from DAMM position into token_vault.
4. claim_to_recipient immediately transfers 1,000 USD1 from token_vault to admin's chosen recipient_token_account.
5. fee_per_share is never updated. total_funded_fee is never updated. Shareholders receive nothing.
6. Next fund_by_claiming_fee call finds 0 claimable fees in the DAMM position.

claimed_amount is never recorded in any vault state field (line 148-165). The only record is the emitted EvtClaimToRecipient event. If events are missed, the extraction is invisible from the on-chain state.

Impact

Since ADMIN is a trusted entity, it's a centralization issue coupled with the logic of claim_to_recipient being able to front-run shareholders.

Recommendation

Remove ClaimPositionFee from CLAIM_FEE_ACTIONS so the two paths never compete for the same fee source:



```
pub static CLAIM_FEE_ACTIONS: [(Pubkey, &[u8], usize); 6] = [  
  // Keep only actions NOT in WHITELISTED_ACTIONS  
  (damm_v2::ID, ClaimReward::DISCRIMINATOR, 5),  
  (dynamic_bonding_curve::ID, CreatorWithdrawSurplus::DISCRIMINATOR, 3),  
  (dynamic_bonding_curve::ID, ClaimCreatorTradingFee::DISCRIMINATOR, 3),  
  (dynamic_bonding_curve::ID, PartnerWithdrawSurplus::DISCRIMINATOR, 3),  
  (dynamic_bonding_curve::ID, WithdrawMigrationFee::DISCRIMINATOR, 3),  
  (dynamic_bonding_curve::ID, ClaimTradingFee::DISCRIMINATOR, 4),  
];
```

Additionally, validate recipient against authorized addresses:

```
let recipient_owner = ctx.accounts.recipient_token_account.owner;  
require!(  
  recipient_owner == STAKERS_RECIPIENT  
  || recipient_owner == TREASURY_RECIPIENT  
  || recipient_owner == LP_RECIPIENT  
  || fee_vault.users.iter().any(|u| u.address == recipient_owner),  
  FeeVaultError::InvalidRecipientAccount  
);
```

If the desired functionality is needed then it should be properly documented in the docs.



Informational Issues

UncheckedAccount Used for FeePolicyConfig Instead of AccountLoader, Missing Discriminator and Owner Validation

Resolved

Path

ix_activate_crack_mode_window.rs
ix_fund_by_claiming_fee.rs

Description

In ix_activate_crack_mode_window.rs:32 and ix_fund_by_claiming_fee.rs:30, fee_policy_config is declared as UncheckedAccount instead of AccountLoader<'info, FeePolicyConfig>. The handler manually replicates what AccountLoader provides – borrowing data, checking length, deserializing via bytemuck::from_bytes.

This introduces unnecessary complexity where as AccountLoader would provide the same functionality with less code. The handler also does not validate the discriminator or owner, which is a defense-in-depth gap.

Impact

The pubkey match provides strong protection. Defense-in-depth gap only.

Recommendation

Replace with AccountLoader<'info, FeePolicyConfig> in both instructions, which validates discriminator and program owner automatically and is much cleaner.



Unsafe Arithmetic in set_fee_policy_config Settlement, Inconsistent With Safe Patterns

Resolved

Path

ix_set_fee_policy_config.rs

Description

In ix_set_fee_policy_config.rs:42-58, the fee settlement logic uses three unsafe patterns that differ from the safe implementations used everywhere else in the codebase:

```
// Line 50 - raw u128 multiplication (no U256-safe mul_shr)
let pending_fee = ((user.share as u128) * reward_per_share_delta) >> 64;

// Lines 51-54 - truncating cast + unwrap_or(0) wipes fees on overflow
user.pending_unclaimed_fee = user
    .pending_unclaimed_fee
    .checked_add(pending_fee as u64)
    .unwrap_or(0);
```

Compare with update_shares() (fee_vault.rs:167-183) which uses mul_shr() (U256-safe), .try_into().map_err(...) (checked cast), and .safe_add() (reverts on overflow). The unwrap_or(0) is particularly concerning, on overflow it silently sets pending_unclaimed_fee to zero instead of reverting.

In practice, overflow requires total_share = 1 with ~80K+ USD1 funded without claims, an edge case that is unrealistic with production vault configurations (4 recipients, typical total_share 100-150).

Impact

Code quality / consistency issue. Practically unreachable with realistic share distributions, but the unsafe patterns deviate from the codebase's safety conventions.

Recommendation

Replace with the same safe pattern used in update_shares() for consistency:

```
if reward_per_share_delta > 0 {
    let pending_fee: u64 = mul_shr(user.share.into(), reward_per_share_delta,
PRECISION_SCALE)
        .ok_or_else(|| FeeVaultError::MathOverflow)?
        .try_into()
        .map_err(|_| FeeVaultError::MathOverflow)?;
    user.pending_unclaimed_fee =
user.pending_unclaimed_fee.safe_add(pending_fee)?;
}
```



DBC (is_dbc) Code Path in fund_by_claiming_fee Is Unreachable Dead Code

Acknowledged

Path

ix_fund_by_claiming_fee.rs

Description

In ix_fund_by_claiming_fee.rs:157, is_dbc checks if source_program == DYNAMIC_BONDING_CURVE_ID. This is always false because WHITELISTED_ACTIONS (constants.rs:48-54) only contains damm_v2::ClaimPositionFee. The is_support_action() check at line 76 rejects any DBC call before the is_dbc check is reached.

DBC actions exist in CLAIM_FEE_ACTIONS (for claim_to_recipient) but not in WHITELISTED_ACTIONS (for fund_by_claiming_fee).

Three unreachable code branches: lines 192-193 (0u128 market cap), lines 237-238 (0u8 range index), and the implicit DBC handling in market cap calculation.

Impact

No security impact. Dead code that may mislead auditors and maintainers into thinking DBC integration is active in the fee-sharing path.

Recommendation

Either remove DBC branches (if not intended for this path) or add DBC actions to WHITELISTED_ACTIONS (if intended but incomplete).

Developer Response

The requirement has changed: DBC is no longer shareable, so I only change whitelist action to minimize change.



Redundant DuplicateUserAddress Check on Same Call Path

Resolved

Path

ix_initialize_fee_vault.rs, fee_vault.rs

Description

The duplicate user address validation is performed twice on the same call path during vault initialization:

Location 1: `InitializeFeeVaultParameters::validate()`

(ix_initialize_fee_vault.rs:77-82):

```
for j in 0..I {
    require!(
        self.users[I].address.ne(&self.users[j].address),
        FeeVaultError::DuplicateUserAddress
    );
}
```

Location 2: `FeeVault::initialize()` (fee_vault.rs:93-98):

```
for j in 0..I {
    require!(
        users[I].address.ne(&users[j].address),
        FeeVaultError::DuplicateUserAddress
    );
}
```

Call path: handle_initialize_fee_vault() → create_fee_vault() → params.validate() (Location 1) → fee_vault.initialize() (Location 2). The $O(n^2)$ loop runs twice on the same data for the same check.

Additionally, FeePolicyConfig::validate() (fee_policy_config.rs:67-128) is never called anywhere in the codebase — it contains mode, phase count, range ordering, and share validation logic that duplicates checks in handle_initialize_fee_policy_config, but since it's never invoked, it's dead code.

Impact

No security impact. Wasted compute units from duplicate $O(n^2)$ loop. Dead validate() method may drift out of sync with actual validation logic.

Recommendation

Remove the duplicate check from FeeVault::initialize() (Location 2) since params.validate() is always called first in create_fee_vault(). Either call FeePolicyConfig::validate() where needed or remove it to avoid maintenance burden.



Closing Summary

In this report, we have considered the security of the American Fun Contract. We performed our audit according to the procedure described above.

2 issues of High severity, 3 issues of Low severity & 4 issues of informational severity were found. The American Fun Team Acknowledged 2 issues and resolved the rest.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With eight years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

April 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com